

Fixing Balloon Pop Hit Detection

<https://gemini.google.com/app/0f0ec9ffc14717dd>

*User prompt: I'm working on a Pygame balloon pop game. In check_pop, clicks only register near the exact center of a balloon. I think the problem is that it compares distance_squared with balloon.radius instead of radius ** 2, so the effective hit radius is only about $\sqrt{\text{radius}}$ pixels. Please fix it by comparing against the squared radius (no sqrt needed). Also, when balloons overlap it returns the oldest balloon, which is drawn underneath. Make it check the most recently drawn balloon first. Show only the changed function.*

Response: Here is the updated function for click_detection.py:

```
def check_pop(balloons, click_pos):
    """
    Returns the balloon that was clicked, or None if the click missed
    every balloon.
    """
    for balloon in reversed(balloons):
        dx = click_pos[0] - balloon.x
        dy = click_pos[1] - balloon.y
        distance_squared = dx * dx + dy * dy
        if distance_squared <= balloon.radius ** 2:
            return balloon
    return None
```

User prompt: Now add balloon types. Define them in one place as a dict/config: normal (red, +10 points, ~70% spawn chance), bonus (gold, +30, ~15%, slightly smaller and faster so it's harder to hit), penalty (dark grey, -20 points, ~15%). Store the type on the Balloon object (kind, color, points). Choose the type using random.choices with weights in _spawn_balloon, and use the balloon's own points in handle_click. Draw a small "+" or "-" label on bonus/penalty balloons so they're distinguishable without relying only on color. Keep logic in game_engine/balloon and drawing in renderer.

Response: **balloon.py**

```
"""
Balloon: falls from the top of the screen. The player must pop it
before it reaches the bottom. Balloons vary in size - this matters for
how click detection should work.
"""

import pygame

class Balloon:
    def __init__(self, x, y, radius, speed, color, kind="normal", points=10):
        self.x = x
        self.y = y
        self.radius = radius
        self.speed = speed
        self.color = color
        self.kind = kind
        self.points = points

    def update(self):
        self.y += self.speed

    def is_past_bottom(self, height):
        return self.y - self.radius > height

    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.radius), int(self.y - self.radius),
            self.radius * 2, self.radius * 2,
        )
```

game_engine.py

```
"""
GameEngine: owns all balloons, spawns new ones, and handles clicks.
"""

import random

from game.balloon import Balloon
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45
```

```

BALLOON_TYPES = {
    "normal": {"color": (220, 40, 40), "points": 10, "weight": 70},
    "bonus": {"color": (255, 215, 0), "points": 30, "weight": 15},
    "penalty": {"color": (100, 100, 100), "points": -20, "weight": 15},
}

class GameEngine:
    def __init__(self):
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0

    def _spawn_balloon(self):
        kinds = list(BALLOON_TYPES.keys())
        weights = [v["weight"] for v in BALLOON_TYPES.values()]
        kind = random.choices(kinds, weights=weights)[0]
        config = BALLOON_TYPES[kind]

        if kind == "bonus":
            radius = random.randint(12, 30)
            speed = random.uniform(2.5, 4.5)
        else:
            radius = random.randint(16, 44)
            speed = random.uniform(1.5, 3.0)

        x = random.randint(radius + 10, WIDTH - radius - 10)

        self.balloons.append(Balloon(
            x=x, y=-radius, radius=radius, speed=speed,
            color=config["color"], kind=kind, points=config["points"]
        ))

    def handle_click(self, pos):
        popped = check_pop(self.balloons, pos)
        if popped is not None:
            self.balloons.remove(popped)
            self.score += popped.points

    def update(self):
        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_balloon()
            self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

        for b in self.balloons:
            b.update()

        self.balloons = [b for b in self.balloons if not b.is_past_bottom(HEIGHT)]

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.balloons, font)
        renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))

```

renderer.py

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (200, 230, 245)
COLOR_TEXT = (30, 30, 30)

def draw_scene(surface, balloons, font=None):
    surface.fill(COLOR_BG)
    for b in balloons:
        pygame.draw.circle(surface, b.color, (int(b.x), int(b.y)), b.radius)
        pygame.draw.line(surface, (120, 120, 120), (b.x, b.y + b.radius), (b.x, b.y + b.radius + 12), 2)

        if font and b.kind != "normal":
            symbol = "+" if b.kind == "bonus" else "-"

```

```

        text_surf = font.render(symbol, True, (255, 255, 255))
        rect = text_surf.get_rect(center=(int(b.x), int(b.y)))
        surface.blit(text_surf, rect)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)

```

User prompt: Add a lives system: start with 3 lives. In update(), balloons that pass the bottom are currently filtered out silently. Instead, count how many escaped and subtract a life for each one, except penalty balloons, since letting those fall is the correct play. Popping a balloon must never change lives. Show "Lives: N" at the top-right. When lives reach 0, set a game_over flag that stops spawning, stops balloon movement and ignores clicks. Don't add the end screen yet; that comes in the next step.

Response: game_engine.py

```

"""
GameEngine: owns all balloons, spawns new ones, and handles clicks.
"""

import random

from game.balloon import Balloon
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45

BALLOON_TYPES = {
    "normal": {"color": (220, 40, 40), "points": 10, "weight": 70},
    "bonus": {"color": (255, 215, 0), "points": 30, "weight": 15},
    "penalty": {"color": (100, 100, 100), "points": -20, "weight": 15},
}

class GameEngine:
    def __init__(self):
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0
        self.lives = 3
        self.game_over = False

    def _spawn_balloon(self):
        kinds = list(BALLOON_TYPES.keys())
        weights = [v["weight"] for v in BALLOON_TYPES.values()]
        kind = random.choices(kinds, weights=weights)[0]
        config = BALLOON_TYPES[kind]

        if kind == "bonus":
            radius = random.randint(12, 30)
            speed = random.uniform(2.5, 4.5)
        else:
            radius = random.randint(16, 44)
            speed = random.uniform(1.5, 3.0)

        x = random.randint(radius + 10, WIDTH - radius - 10)

        self.balloons.append(Balloon(
            x=x, y=-radius, radius=radius, speed=speed,
            color=config["color"], kind=kind, points=config["points"]
        ))

    def handle_click(self, pos):
        if self.game_over:
            return

        popped = check_pop(self.balloons, pos)
        if popped is not None:
            self.balloons.remove(popped)
            self.score += popped.points

```

```

def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        self._spawn_balloon()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for b in self.balloons:
        b.update()

    active_balloons = []
    for b in self.balloons:
        if b.is_past_bottom(HEIGHT):
            if b.kind != "penalty":
                self.lives -= 1
        else:
            active_balloons.append(b)

    self.balloons = active_balloons

    if self.lives <= 0:
        self.lives = 0
        self.game_over = True

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.balloons, font)

    # Draw Score (Top-Left)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))

    # Draw Lives (Top-Right)
    lives_text = f"Lives: {self.lives}"
    text_width = font.size(lives_text)[0]
    renderer.draw_text(surface, font, lives_text, (WIDTH - text_width - 10, 10))

```

User prompt: Add a 30-second timed round. Track time with `pygame.time.get_ticks()` rather than counting frames so it's accurate if FPS drops. Show "Time: N" centered at the top. The round ends when time hits 0 OR lives hit 0, whichever comes first. When it ends: stop spawning and accepting clicks, and show the final score with the existing `renderer.draw_banner()` plus a line saying "Press R to play again". Move all starting state (`score`, `lives`, `balloons`, `start time`, `game_over`) into a `reset()` method that `__init__` also calls, and handle the R key in `main.py`'s event loop to call it. Show the full updated `game_engine.py` and `main.py`.

Response: `game_engine.py`

```

"""
GameEngine: owns all balloons, spawns new ones, and handles clicks.
"""

import random
import pygame

from game.balloon import Balloon
from game.click_detection import check_pop
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45
ROUND_TIME_SEC = 30

BALLOON_TYPES = {
    "normal": {"color": (220, 40, 40), "points": 10, "weight": 70},
    "bonus": {"color": (255, 215, 0), "points": 30, "weight": 15},
    "penalty": {"color": (100, 100, 100), "points": -20, "weight": 15},
}

class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0

```

```

self.lives = 3
self.game_over = False
self.start_time = pygame.time.get_ticks()
self.time_left = ROUND_TIME_SEC

def _spawn_balloon(self):
    kinds = list(BALLOON_TYPES.keys())
    weights = [v["weight"] for v in BALLOON_TYPES.values()]
    kind = random.choices(kinds, weights=weights)[0]
    config = BALLOON_TYPES[kind]

    if kind == "bonus":
        radius = random.randint(12, 30)
        speed = random.uniform(2.5, 4.5)
    else:
        radius = random.randint(16, 44)
        speed = random.uniform(1.5, 3.0)

    x = random.randint(radius + 10, WIDTH - radius - 10)

    self.balloons.append(Balloon(
        x=x, y=-radius, radius=radius, speed=speed,
        color=config["color"], kind=kind, points=config["points"]
    ))

def handle_click(self, pos):
    if self.game_over:
        return

    popped = check_pop(self.balloons, pos)
    if popped is not None:
        self.balloons.remove(popped)
        self.score += popped.points

def update(self):
    if self.game_over:
        return

    elapsed_ms = pygame.time.get_ticks() - self.start_time
    self.time_left = max(0, ROUND_TIME_SEC - (elapsed_ms // 1000))

    if self.time_left <= 0:
        self.game_over = True

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        self._spawn_balloon()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for b in self.balloons:
        b.update()

    active_balloons = []
    for b in self.balloons:
        if b.is_past_bottom(HEIGHT):
            if b.kind != "penalty":
                self.lives -= 1
        else:
            active_balloons.append(b)

    self.balloons = active_balloons

    if self.lives <= 0:
        self.lives = 0
        self.game_over = True

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.balloons, font)

    # Draw Score (Top-Left)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))

    # Draw Time (Top-Center)
    time_text = f"Time: {self.time_left}"
    time_width = font.size(time_text)[0]

```

```

renderer.draw_text(surface, font, time_text, (WIDTH // 2 - time_width // 2, 10))

# Draw Lives (Top-Right)
lives_text = f"Lives: {self.lives}"
lives_width = font.size(lives_text)[0]
renderer.draw_text(surface, font, lives_text, (WIDTH - lives_width - 10, 10))

if self.game_over:
    renderer.draw_banner(surface, font, f"Final Score: {self.score}")

    # Draw restart prompt below the banner
    restart_text = "Press R to play again"
    restart_width = font.size(restart_text)[0]
    renderer.draw_text(surface, font, restart_text, (WIDTH // 2 - restart_width // 2, HEIGHT // 2 + 35))

```

main.py

```

"""

```

```

Balloon Pop (Lab Starter)

```

```

Run with: python3 main.py

```

```

Click balloons to pop them before they reach the bottom.
"""

```

```

import pygame

```

```

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

```

```

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Balloon Pop")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 22)

    engine = GameEngine()
    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN:
                engine.handle_click(event.pos)
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r:
                    engine.reset()

        engine.update()
        engine.draw(screen, font)

        pygame.display.flip()
        clock.tick(60)

    pygame.quit()

```

```

if __name__ == "__main__":
    main()

```